

AWS Cloud Infrastructure Architecture and Implementation

SECTION 1 — EVOLUTION OF INFRASTRUCTURE AND CLOUD COMPUTING

1.1 Introduction

Initially, organizations hosted applications using traditional physical infrastructure. Companies maintain physical servers, storage devices, networking hardware, and data centers to run applications and services. Managing this infrastructure requires high investment, hardware maintenance, cooling systems, backup management, and dedicated infrastructure teams.

Over time, organizations faced challenges such as high operational cost, scalability limitations, maintenance complexity, and disaster recovery issues. To solve these problems, cloud computing evolved as a modern infrastructure solution.

Cloud computing introduced on-demand infrastructure, scalability, automation, high availability, and global accessibility. Organizations can now provision virtual resources through cloud providers such as AWS instead of maintaining physical hardware.

1.2 Traditional Infrastructure Problems

Traditional infrastructure introduced several challenges:

- High hardware cost
- Manual maintenance
- Difficult scalability
- Downtime risks
- Complex disaster recovery
- Resource underutilization

These challenges increased operational complexity for organizations.

1.3 Introduction to Cloud Computing

Cloud computing is the delivery of computing services such as servers, storage, networking, databases, and security over the internet.

Cloud platforms provide virtual infrastructure resources on demand. Organizations can deploy applications quickly without purchasing physical hardware.

Benefits of cloud computing include:

- Scalability
- Elasticity
- Cost optimization
- High availability
- Security
- Global accessibility

SECTION 2 — INTRODUCTION TO CLOUD COMPUTING

2.1 What is Cloud Computing

Cloud computing allows organizations to use virtual infrastructure resources over the internet. Instead of managing physical servers, organizations can provision resources whenever required using cloud providers such as AWS.

Cloud computing simplifies infrastructure management and improves application speed.

2.2 Cloud Service Models

Infrastructure as a Service (IaaS)

Provides virtual infrastructure resources.

Examples:

- Amazon EC2
- Amazon VPC
- Amazon EBS

Platform as a Service (PaaS)

Provides platforms for application development and deployment.

Example:

- AWS Elastic Beanstalk

Software as a Service (SaaS)

Provides software applications through the internet.

Examples:

- Gmail
- Office 365

2.3 Traditional Infrastructure vs Cloud Infrastructure

Traditional Infrastructure	Cloud Infrastructure
Physical hardware required	Virtual infrastructure
High cost	Pay-as-you-go
Manual scaling	Automatic scaling
Hardware maintenance required	Managed by cloud provider
Limited scalability	Highly scalable

SECTION 3 — AWS GLOBAL INFRASTRUCTURE

3.1 AWS Regions

AWS Regions are geographical locations containing multiple data centers.

Examples:

- ap-south-1 (Mumbai)
- us-east-1 (N. Virginia)
- eu-west-1 (Ireland)

Organizations select regions based on latency, compliance, and customer location.

3.2 Availability Zones

Availability Zones are isolated data centers within a region.

Deploying resources across multiple Availability Zones improves:

- High Availability
- Fault Tolerance
- Disaster Recovery

3.3 Edge Locations

Edge Locations are AWS locations used for low-latency content delivery.

Services such as CloudFront and Route53 use Edge Locations to improve performance.

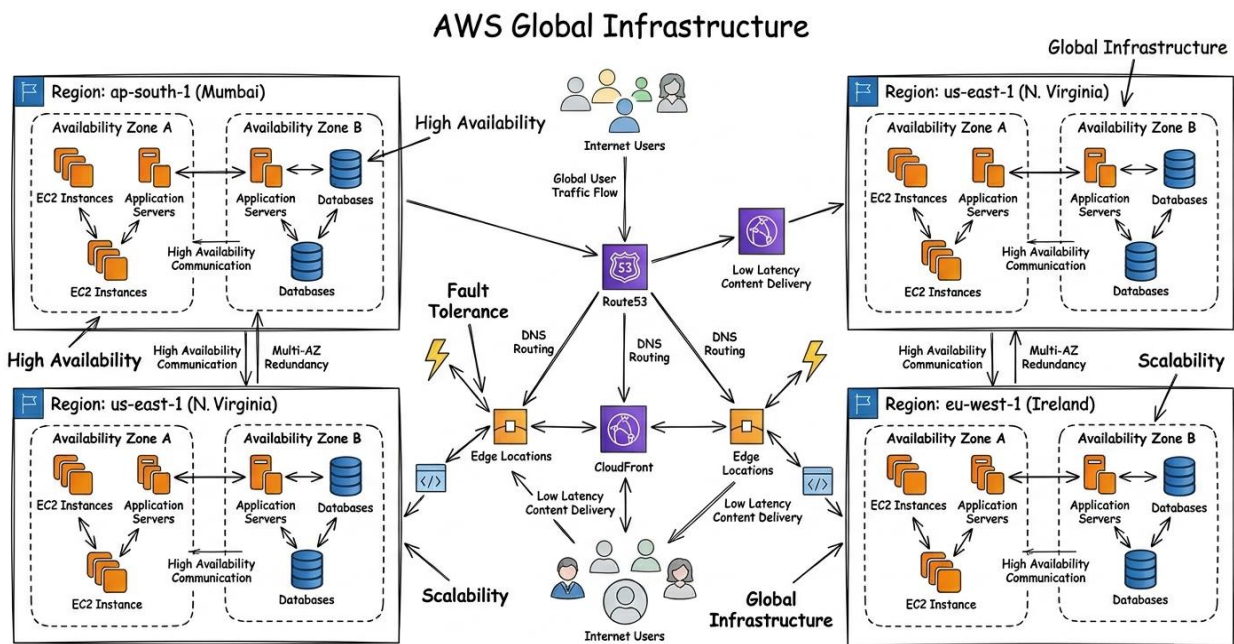


Figure 3.1 — AWS Global Infrastructure Architecture

SECTION 4 — NETWORKING FUNDAMENTALS

4.1 Introduction

Networking is one of the most important components in cloud computing. It enables communication between users, servers, applications, and databases inside the infrastructure environment.

In AWS, networking services are used to build secure, scalable, and isolated cloud architectures. Proper networking configuration helps organizations manage traffic flow, improve security, and maintain high availability.

Networking concepts such as IP addressing, CIDR blocks, subnets, DNS, ports, and routing are essential for designing production-grade AWS infrastructure.

4.2 IP Addressing

An IP address is a unique address assigned to a device for communication inside a network.

AWS mainly uses two types of IP addresses:

Public IP Address

A Public IP address allows communication through the internet. Resources such as web servers and load balancers use public IP addresses to communicate with external users.

Examples:

- Application Load Balancer
- Bastion Host
- Public Web Servers

Private IP Address

A Private IP address is used for internal communication within the VPC. Private IP addresses cannot be accessed directly through the internet.

Examples:

- Application Servers
- Database Servers

Private IP communication improves infrastructure security.

4.3 CIDR Block

CIDR (Classless Inter-Domain Routing) is used to define IP address ranges inside a network.

Example:

10.0.0.0/16

In this example:

- 10.0.0.0 represents the network address
- /16 represents the subnet mask

CIDR blocks are used while creating VPCs and subnets in AWS.

4.4 Subnetting

Subnetting is the process of dividing a network into smaller networks called subnets.

AWS mainly uses two types of subnets:

Public Subnet

Public subnets contain resources that require internet access.

Examples:

- Load Balancer

- Bastion Host
- Frontend Web Servers

Public subnets are connected to the Internet Gateway.

Private Subnet

Private subnets contain internal resources that should not be directly accessible from the internet.

Examples:

- Backend Application Servers
- Database Servers

Private subnets use NAT Gateway for outbound internet communication.

4.5 DNS (Domain Name System)

DNS converts domain names into IP addresses.

Example:

www.example.com → 192.168.1.10

AWS Route53 is used for DNS management in AWS infrastructure.

DNS improves user accessibility because users can access applications using domain names instead of remembering IP addresses.

4.6 Ports and Protocols

Ports are communication endpoints used by applications and services.

Common ports include:

Port	Protocol	Purpose
22	SSH	Remote Server Access
80	HTTP	Web Traffic
443	HTTPS	Secure Web Traffic
3306	MySQL	Database Communication

Protocols define communication rules between systems.

Common protocols:

- TCP
- UDP
- HTTP
- HTTPS
- SSH

4.7 Traffic Flow in AWS Infrastructure

In a production AWS architecture:

- Users access applications through the internet
- Traffic reaches the Load Balancer
- Load Balancer forwards requests to Web Tier
- Web Tier communicates with App Tier
- App Tier communicates with Database Tier

This layered communication improves:

- Security
- Isolation
- Scalability
- High Availability

4.8 Security in Networking

AWS networking security is implemented using multiple services.

Security Groups

Acts as an instance-level firewall.

Network ACLs

Acts as a subnet-level firewall.

Route Tables

Controls network traffic routing.

Internet Gateway

Provides internet access for public resources.

NAT Gateway

Provides outbound internet access for private resources.

4.9 Summary

Networking fundamentals are essential for building secure and scalable AWS infrastructure. Components such as IP addressing, CIDR blocks, subnets, DNS, ports, and routing mechanisms help establish secure communication between cloud resources.

AWS networking services such as VPC, Security Groups, Route Tables, Internet Gateway, and NAT Gateway help organizations design production-grade cloud architectures efficiently.

SECTION 5 — AMAZON VPC

5.1 Introduction

Amazon Virtual Private Cloud (VPC) is a logically isolated virtual network inside AWS. It allows organizations to securely launch AWS resources such as EC2 instances, databases, load balancers, and application servers within a controlled networking environment.

VPC helps organizations design secure and scalable cloud architectures by controlling IP addressing, subnet configuration, routing, and security policies.

AWS provides complete control over networking components inside the VPC environment.

5.2 Why VPC is Required

VPC is required to create isolated and secure cloud infrastructure environments.

Using VPC helps organizations:

- Control network communication
- Separate public and private resources
- Improve infrastructure security
- Implement custom networking architecture
- Manage routing and traffic flow
- Build production-grade cloud infrastructure

Without VPC, organizations cannot properly manage networking and security inside AWS.

5.3 Components of Amazon VPC

Amazon VPC contains several important networking components.

Subnets

Subnets divide the VPC network into smaller networks.

AWS mainly uses:

- Public Subnets
- Private Subnets

Public subnets host internet-facing resources, while private subnets host internal resources.

Route Tables

Route Tables control traffic routing inside the VPC.

They define how traffic moves between:

- Subnets
- Internet Gateway
- NAT Gateway
- Internal resources

Internet Gateway (IGW)

The Internet Gateway allows communication between public subnet resources and the internet.

Resources inside public subnets use the Internet Gateway for inbound and outbound internet communication.

NAT Gateway

NAT Gateway provides outbound internet access for private subnet resources.

Private instances use NAT Gateway for:

- Package installation
- Software updates

- External API communication

Without exposing the instances directly to the internet.

Security Groups

Security Groups act as instance-level virtual firewalls.

They control:

- Inbound traffic
- Outbound traffic

Security Groups help secure EC2 instances and applications.

Network ACLs (NACLs)

Network ACLs act as subnet-level firewalls.

They provide an additional security layer for subnet traffic filtering.

5.4 Public and Private Subnets

Public Subnet

Public subnets contain internet-facing resources.

Examples:

- Application Load Balancer
- Bastion Host
- Web Servers

Public subnets contain routes connected to the Internet Gateway.

Private Subnet

Private subnets contain internal resources that should not be directly accessible through the internet.

Examples:

- Application Servers
- Database Servers

Private subnets use NAT Gateway for outbound internet communication.

5.5 Default VPC vs Custom VPC

Default VPC	Custom VPC
Automatically created by AWS	Manually created by user
Basic configuration	Fully customizable
Less control	Better security and flexibility
Public subnet by default	Custom subnet design
Suitable for testing	Suitable for production environments

5.6 VPC Communication Flow

In a production architecture:

- Users access applications through the internet
- Traffic enters through Internet Gateway
- Load Balancer forwards traffic to Web Tier
- Web Tier communicates with App Tier
- App Tier communicates with Database Tier

This layered architecture improves:

- Security
- Scalability
- Isolation
- High Availability

5.7 Benefits of Amazon VPC

Amazon VPC provides several advantages.

Secure Networking

Resources are isolated inside a private virtual network.

Traffic Control

Route Tables and Gateways help manage traffic flow.

Scalability

VPC supports scalable infrastructure deployment.

High Availability

Resources can be deployed across multiple Availability Zones.

Flexible Architecture

Organizations can design custom networking environments based on application requirements.

5.8 Summary

Amazon VPC is one of the most important AWS networking services used to build secure, scalable, and production-grade cloud architectures.

Components such as subnets, route tables, Internet Gateway, NAT Gateway, Security Groups, and Network ACLs help organizations manage secure communication and traffic flow inside AWS infrastructure environments.

6.1 Introduction

A custom Amazon VPC was implemented to create a secure and isolated networking environment for the production-grade AWS infrastructure. The custom VPC architecture was designed to separate frontend, backend, and database resources using public and private subnets.

The implementation also included secure routing, internet communication, outbound internet access for private resources, and multi-tier network segmentation.

6.2 Objective

The objective of this implementation was to:

- Create a secure AWS networking environment
- Build isolated public and private networks
- Implement controlled traffic routing
- Support secure communication between application layers
- Build a production-grade cloud architecture

6.3 Case Study

A secure networking architecture was required for deploying a three-tier web application inside AWS.

The frontend application needed internet accessibility for users, while backend application servers and database servers required secure private communication without direct internet exposure.

To achieve this architecture, a custom VPC was designed using multiple subnets, route tables, Internet Gateway, and NAT Gateway.

6.4 AWS Services Used

The following AWS services were used during this implementation:

- Amazon VPC
- Public Subnets
- Private Subnets
- Route Tables
- Internet Gateway

- NAT Gateway
- Security Groups

6.5 VPC Configuration

The following VPC configuration was implemented:

Component	Configuration
VPC CIDR	10.0.0.0/16
Public Subnet 1	10.0.1.0/24
Public Subnet 2	10.0.3.0/24
Private Subnet 1	10.0.2.0/24
Private Subnet 2	10.0.4.0/24

6.6 Architecture Design

The VPC architecture was designed using:

- Multi-AZ deployment
- Public subnet segmentation
- Private subnet isolation
- Controlled internet communication
- Secure backend communication

Public resources were deployed inside public subnets, while backend resources were deployed inside private subnets.

6.7 Implementation Steps

Step 1 — Created Custom VPC

A custom VPC was created using the following CIDR block:

10.0.0.0/16

This CIDR block provided the private networking range for the infrastructure.

Step 2 — Created Public Subnets

Two public subnets were created across multiple Availability Zones.

Public subnets were used for:

- Application Load Balancer
- Bastion Host
- Frontend Web Servers

Step 3 — Created Private Subnets

Two private subnets were created for backend resources.

Private subnets were used for:

- Application Servers
- Database Servers

Private subnets restricted direct internet communication.

Step 4 — Configured Route Tables

Separate route tables were configured for:

- Public Subnets
- Private Subnets

Public route tables used Internet Gateway routing, while private route tables used NAT Gateway routing.

Step 5 — Attached Internet Gateway

An Internet Gateway was created and attached to the VPC to provide internet communication for public subnet resources.

Step 6 — Configured NAT Gateway

A NAT Gateway was deployed inside the public subnet to provide outbound internet access for private subnet resources.

6.8 Traffic Flow

The traffic flow inside the architecture was implemented as follows:

- User traffic entered through the Internet Gateway
- Traffic reached the Application Load Balancer
- Load Balancer forwarded requests to frontend servers
- Frontend servers communicated with backend servers
- Backend servers communicated with database servers privately

This architecture improved:

- Security
- Isolation
- Scalability
- High Availability

6.9 Validation

The following validations were performed successfully:

- Public instances accessed through internet
- Private instances accessed through Bastion Host
- NAT Gateway outbound communication verified
- Route Table routing verified
- Secure communication between tiers verified

6.10 Benefits of Custom VPC

The implemented custom VPC provided several advantages:

- Secure network isolation
- Controlled internet access
- Multi-tier communication
- Production-grade architecture
- Better scalability
- Improved infrastructure security

6.11 Summary

A production-grade custom VPC architecture was successfully implemented using public subnets, private subnets, route tables, Internet Gateway, and NAT Gateway.

The implementation provided secure communication, network isolation, scalable infrastructure, and high availability required for enterprise cloud environments.

SECTION 7 — TASK 2: PUBLIC AND PRIVATE SUBNET CONFIGURATION

7.1 Introduction

Public and private subnets were configured inside the custom VPC to separate internet-facing resources from internal backend resources. This subnet segregation improved infrastructure security, controlled traffic flow, and supported production-grade architecture implementation.

The subnet design was implemented across multiple Availability Zones to ensure high availability and fault tolerance.

7.2 Objective

The objective of this implementation was to:

- Separate frontend and backend resources
- Restrict direct internet access to internal resources
- Improve security using subnet isolation
- Support secure communication between application layers
- Implement multi-AZ deployment architecture

7.3 Public Subnet Configuration

Public subnets were created for internet-facing resources.

The following resources were deployed inside public subnets:

- Application Load Balancer
- Bastion Host
- Frontend Web Servers
- NAT Gateway

Public subnets contained route table entries connected to the Internet Gateway for internet communication.

7.4 Private Subnet Configuration

Private subnets were created for internal backend resources.

The following resources were deployed inside private subnets:

- Backend Application Servers
- Database Servers

Private subnets did not allow direct internet access. Outbound internet communication was provided using NAT Gateway.

This implementation improved infrastructure security by preventing external access to sensitive backend resources.

7.5 Subnet Configuration Details

Subnet Type	CIDR Block	Purpose
Public Subnet 1	10.0.1.0/24	Frontend Resources
Public Subnet 2	10.0.3.0/24	Multi-AZ Frontend Resources
Private Subnet 1	10.0.2.0/24	Backend Resources
Private Subnet 2	10.0.4.0/24	Multi-AZ Backend Resources

7.6 Multi-AZ Deployment

The subnet architecture was implemented across multiple Availability Zones.

Benefits of Multi-AZ deployment include:

- High Availability
- Fault Tolerance
- Disaster Recovery
- Continuous Application Availability

If one Availability Zone becomes unavailable, resources inside another Availability Zone continue serving application traffic.

7.7 Traffic Flow

The communication flow inside the subnet architecture was implemented as follows:

- Internet users access frontend resources through public subnets
- Application Load Balancer forwards traffic to web servers
- Web servers communicate with backend application servers privately
- Backend servers communicate with database servers inside private subnets

This architecture provided secure layered communication inside the infrastructure.

7.8 Security Implementation

Security was implemented using:

Security Groups

Controlled inbound and outbound traffic for EC2 instances.

Network ACLs

Controlled subnet-level traffic filtering.

Private Subnet Isolation

Restricted direct internet communication for backend resources.

7.9 Validation

The following validations were performed successfully:

- Public instances accessible through internet
- Private instances inaccessible from internet
- Backend communication verified
- NAT Gateway outbound communication verified
- Multi-AZ subnet deployment verified

7.10 Benefits of Public and Private Subnet Architecture

The implemented subnet architecture provided several advantages:

- Better infrastructure security
- Network isolation
- Controlled internet access
- Scalable architecture
- High Availability
- Production-grade deployment

7.11 Summary

Public and private subnets were successfully configured inside the custom VPC to separate frontend and backend resources securely.

The implementation improved infrastructure security, controlled traffic flow, enabled multi-tier communication, and supported high availability for the production-grade AWS architecture.

SECTION 8 — TASK 3: INTERNET GATEWAY CONFIGURATION

8.1 Introduction

An Internet Gateway (IGW) was configured to enable internet communication for resources deployed inside the public subnets. The Internet Gateway acts as a bridge between the Amazon VPC and the public internet.

It allows inbound and outbound internet traffic for internet-facing resources such as Application Load Balancers, Bastion Hosts, and frontend web servers.

8.2 Objective

The objective of this implementation was to:

- Enable internet access for public subnet resources
- Allow inbound and outbound internet communication
- Support public-facing applications
- Provide external user accessibility
- Implement secure internet routing

8.3 Internet Gateway Overview

An Internet Gateway is a highly available AWS networking component attached to a VPC.

Functions of Internet Gateway include:

- Provides internet connectivity
- Enables public communication
- Supports public IP routing
- Allows inbound and outbound traffic

Without an Internet Gateway, public subnet resources cannot communicate with the internet.

8.4 Implemented Components

The following components were configured during this implementation:

- Internet Gateway
- VPC Attachment
- Public Route Table
- Internet Routing

8.5 Implementation Steps

Step 1 — Created Internet Gateway

An Internet Gateway was created inside AWS VPC services.

The Internet Gateway acts as the entry and exit point for internet traffic.

Step 2 — Attached Internet Gateway to VPC

The Internet Gateway was attached to the custom VPC.

This attachment enabled internet communication for resources inside the VPC.

Step 3 — Configured Public Route Table

The public route table was configured with the following route:

`0.0.0.0/0 → Internet Gateway`

This route allowed internet-bound traffic from public subnets to pass through the Internet Gateway.

Step 4 — Associated Public Subnets

Public subnets were associated with the public route table.

This enabled internet communication for:

- Application Load Balancer
- Bastion Host

- Frontend Web Servers

8.6 Traffic Flow

The traffic flow was implemented as follows:

- Internet users send requests
- Requests enter through Internet Gateway
- Traffic reaches public subnet resources
- Public resources send responses back through Internet Gateway

This enabled secure communication between external users and AWS infrastructure.

8.7 Security Considerations

The following security mechanisms were implemented:

Security Groups

Controlled inbound and outbound traffic for public resources.

Public Subnet Isolation

Only required resources were placed inside public subnets.

Controlled Port Access

Only necessary ports such as:

- 80 (HTTP)
- 443 (HTTPS)
- 22 (SSH)

were allowed.

8.8 Validation

The following validations were performed successfully:

- Public instances accessible through internet
- Application Load Balancer reachable publicly
- Bastion Host SSH access verified

- Internet communication validated
- Public routing verified

8.9 Benefits of Internet Gateway

The implemented Internet Gateway provided several advantages:

- Internet connectivity
- Public application accessibility
- Secure internet routing
- High availability
- Support for frontend communication

8.10 Summary

An Internet Gateway was successfully configured and attached to the custom VPC to provide internet communication for public subnet resources.

The implementation enabled secure inbound and outbound internet traffic, supported public-facing applications, and established proper internet routing within the production-grade AWS infrastructure.

SECTION 9 — TASK 4: NAT GATEWAY CONFIGURATION

9.1 Introduction

A NAT Gateway was configured to provide outbound internet access for resources deployed inside private subnets. The NAT Gateway allows private instances to access the internet for software updates, package installation, and external communication without exposing those instances directly to the public internet.

This implementation improved infrastructure security by keeping backend resources private while still allowing necessary outbound communication.

9.2 Objective

The objective of this implementation was to:

- Provide outbound internet access for private subnet resources
- Restrict direct internet exposure for backend servers

- Improve infrastructure security
- Support package installation and updates
- Maintain secure private communication

9.3 NAT Gateway Overview

A NAT (Network Address Translation) Gateway is an AWS-managed networking service used for outbound internet communication from private subnets.

The NAT Gateway is deployed inside a public subnet and connected to an Elastic IP address.

Private instances send internet-bound traffic through the NAT Gateway, while inbound internet traffic from external users remains blocked.

9.4 Implemented Components

The following components were configured:

- NAT Gateway
- Elastic IP
- Public Subnet Deployment
- Private Route Table Configuration

9.5 Implementation Steps

Step 1 — Allocated Elastic IP

An Elastic IP address was allocated for the NAT Gateway.

Elastic IP provides a static public IP address for internet communication.

Step 2 — Created NAT Gateway

The NAT Gateway was created inside the public subnet.

The NAT Gateway used the allocated Elastic IP for outbound internet connectivity.

Step 3 — Configured Private Route Table

The private route table was configured with the following route:

0.0.0.0/0 → NAT Gateway

This route allowed private subnet resources to send outbound traffic through the NAT Gateway.

Step 4 — Associated Private Subnets

Private subnets were associated with the private route table.

This enabled outbound internet communication for:

- Application Servers
- Database-related services
- Backend resources

9.6 Traffic Flow

The communication flow was implemented as follows:

- Private instances initiate outbound requests
- Traffic reaches NAT Gateway
- NAT Gateway forwards traffic to the internet
- Responses return through NAT Gateway
- External users cannot directly access private instances

This architecture maintained secure backend communication.

9.7 Security Benefits

The NAT Gateway implementation improved infrastructure security through:

Private Resource Isolation

Backend resources remained inaccessible from the internet.

Controlled Internet Access

Only outbound communication was allowed.

Reduced Attack Surface

Private instances did not require public IP addresses.

9.8 Validation

The following validations were performed successfully:

- Private instances accessed internet successfully
- Package installation verified
- Software updates completed successfully
- Direct internet access to private instances restricted
- NAT routing verified

9.9 Advantages of NAT Gateway

The implemented NAT Gateway provided several advantages:

- Secure outbound internet access
- Better backend security
- Simplified architecture
- High availability
- Managed AWS networking service

9.10 Summary

A NAT Gateway was successfully configured to provide secure outbound internet communication for private subnet resources.

The implementation ensured that backend servers remained private while still supporting internet access for updates, package installations, and external communication within the production-grade AWS infrastructure.

SECTION 10 — TASK 5: ROUTE TABLE CONFIGURATION

10.1 Introduction

Route Tables were configured to control network traffic flow inside the Amazon VPC. Route Tables determine how traffic moves between subnets, Internet Gateway, NAT Gateway, and internal AWS resources.

Separate route tables were implemented for public and private subnets to maintain secure communication and proper traffic routing within the production-grade infrastructure.

10.2 Objective

The objective of this implementation was to:

- Control traffic routing inside the VPC
- Separate public and private communication
- Enable internet access for public resources
- Enable outbound internet access for private resources
- Implement secure network routing

10.3 Route Table Overview

A Route Table contains a set of rules called routes that determine where network traffic should be directed.

Each subnet inside the VPC must be associated with a Route Table.

AWS mainly used:

- Public Route Table
- Private Route Table

10.4 Public Route Table Configuration

The Public Route Table was configured for public subnets.

It contained the following route:

0.0.0.0/0 → Internet Gateway

This route allowed internet-bound traffic to pass through the Internet Gateway.

Public subnets associated with this route table included:

- Frontend Web Servers
- Bastion Host
- Application Load Balancer

10.5 Private Route Table Configuration

The Private Route Table was configured for private subnets.

It contained the following route:

`0.0.0.0/0 → NAT Gateway`

This route allowed private subnet resources to access the internet through the NAT Gateway.

Private subnets associated with this route table included:

- Backend Application Servers
- Database-related resources

10.6 Route Table Associations

Subnet associations were configured as follows:

Subnet Type	Associated Route Table
Public Subnets	Public Route Table
Private Subnets	Private Route Table

This ensured proper traffic routing for each subnet type.

10.7 Traffic Flow

The traffic flow inside the VPC was implemented as follows:

Public Traffic Flow

- User traffic enters through Internet Gateway
- Public Route Table forwards traffic to public resources
- Public resources respond back through Internet Gateway

Private Traffic Flow

- Private resources initiate outbound requests
- Private Route Table forwards traffic to NAT Gateway
- NAT Gateway sends traffic to internet

- Responses return securely through NAT Gateway

10.8 Security Implementation

Route Tables improved infrastructure security through:

Network Segmentation

Public and private traffic were separated properly.

Controlled Routing

Only required traffic paths were configured.

Secure Backend Communication

Private resources remained inaccessible directly from the internet.

10.9 Validation

The following validations were performed successfully:

- Public routing verified
- Internet Gateway routing verified
- NAT Gateway routing verified
- Public instances accessible through internet
- Private instances accessed internet securely through NAT Gateway

10.10 Benefits of Route Tables

The implemented Route Tables provided several advantages:

- Controlled traffic flow
- Better network management
- Improved infrastructure security
- Public and private network separation
- Secure internet communication

10.11 Summary

Route Tables were successfully configured to manage secure traffic routing inside the Amazon VPC.

Separate public and private route tables enabled controlled internet communication, secure backend access, and proper traffic segmentation within the production-grade AWS infrastructure.

SECTION 11 — EC2 INSTANCE DEPLOYMENT

11.1 Introduction

Amazon EC2 (Elastic Compute Cloud) instances were deployed to host frontend and backend application services inside the AWS infrastructure. EC2 provides scalable virtual servers that allow organizations to run applications in a secure and flexible cloud environment.

The deployment included multiple EC2 instances distributed across Availability Zones to support high availability and fault tolerance.

11.2 Objective

The objective of this implementation was to:

- Deploy scalable compute resources
- Host frontend and backend applications
- Support high availability architecture
- Implement secure server communication
- Build production-grade application infrastructure

11.3 EC2 Overview

Amazon EC2 is a virtual server service provided by AWS.

EC2 instances are used for:

- Hosting applications
- Running web servers
- Backend processing
- Database communication
- Application deployment

EC2 provides:

- Scalability
- Flexibility
- High Availability
- Custom server configuration

11.4 Implemented Components

The following EC2 resources were deployed:

- Frontend Web Servers
- Backend Application Servers
- Bastion Host Server

The deployment was implemented across multiple Availability Zones.

11.5 EC2 Configuration

The following configurations were used during deployment:

Component	Configuration
Operating System	Ubuntu Linux
Instance Type	t2.micro
Authentication	SSH Key Pair
Security	Security Groups
Networking	VPC Subnets

11.6 Frontend Web Server Deployment

Frontend EC2 instances were deployed inside public subnets.

These servers hosted:

- Static Web Applications
- NGINX Web Server
- Frontend Services

Frontend instances were connected to the Application Load Balancer for traffic distribution.

11.7 Backend Application Server Deployment

Backend EC2 instances were deployed inside private subnets.

These servers handled:

- Backend Application Logic
- Internal Communication
- Database Connectivity

Private deployment improved infrastructure security by restricting direct internet access.

11.8 Bastion Host Deployment

A Bastion Host EC2 instance was deployed inside the public subnet.

The Bastion Host was used to:

- Access private instances securely
- Perform administrative operations
- Enable SSH jump access

This implementation improved secure server management.

11.9 Security Configuration

Security was implemented using Security Groups.

The following communication rules were configured:

Source	Destination	Purpose
Internet	ALB	HTTP/HTTPS Access
Bastion Host	Private EC2	SSH Access
ALB	Web Servers	Application Traffic
Web Tier	App Tier	Backend Communication

Only required ports were allowed.

11.10 Traffic Flow

The traffic flow inside the EC2 architecture was implemented as follows:

- Users access applications through Application Load Balancer
- ALB forwards traffic to frontend EC2 instances
- Frontend instances communicate with backend EC2 instances
- Backend instances process application requests

This layered architecture improved scalability and security.

11.11 Validation

The following validations were performed successfully:

- EC2 instances launched successfully
- SSH access verified
- Web application deployment verified
- Backend communication verified
- Load Balancer connectivity verified

11.12 Benefits of EC2 Deployment

The implemented EC2 architecture provided several advantages:

- Scalable compute infrastructure
- Flexible application deployment
- Secure communication
- High availability
- Production-grade application hosting

11.13 Summary

Amazon EC2 instances were successfully deployed to host frontend and backend application services inside the AWS infrastructure.

The deployment supported scalable application hosting, secure communication, high availability, and production-grade cloud architecture implementation.

SECTION 12 — BASTION HOST SETUP

12.1 Introduction

A Bastion Host was implemented to provide secure administrative access to EC2 instances deployed inside private subnets. Since private instances do not have direct internet access, the Bastion Host acts as a secure jump server for SSH connectivity.

The Bastion Host was deployed inside the public subnet and configured with controlled SSH access to improve infrastructure security.

12.2 Objective

The objective of this implementation was to:

- Provide secure SSH access to private EC2 instances
- Restrict direct internet access to backend resources
- Implement secure administrative communication
- Improve infrastructure security
- Support production-grade server management

12.3 Bastion Host Overview

A Bastion Host is a secure EC2 instance deployed inside a public subnet.

Administrators first connect to the Bastion Host using SSH and then access private EC2 instances from the Bastion Host.

This architecture prevents direct internet exposure of backend servers.

12.4 Implemented Components

The following components were configured:

- Bastion Host EC2 Instance
- Public Subnet Deployment
- SSH Key Pair Authentication
- Security Group Rules
- Private EC2 SSH Access

12.5 Bastion Host Configuration

Component	Configuration
Instance Type	t2. micro
Operating System	Ubuntu Linux
Deployment Location	Public Subnet
Access Method	SSH
Authentication	Key Pair

12.6 Security Group Configuration

The Bastion Host Security Group allowed:

Port	Protocol	Purpose
22	SSH	Administrative Access

SSH access was restricted to trusted IP addresses for security purposes.

Private EC2 Security Groups allowed SSH access only from the Bastion Host.

12.7 SSH Communication Flow

The communication flow was implemented as follows:

- Administrator connects to Bastion Host using SSH
- Bastion Host establishes SSH connection to private EC2 instances
- Administrative tasks are performed securely

This architecture ensured secure backend server access.

12.8 Security Benefits

The Bastion Host implementation improved infrastructure security through:

Private Instance Isolation

Private instances were not exposed directly to the internet.

Controlled Administrative Access

Only authorized administrators could access backend servers.

Reduced Attack Surface

Direct SSH access to private instances was restricted.

12.9 Validation

The following validations were performed successfully:

- Bastion Host accessible through SSH
- SSH access to private instances verified

- Backend instances inaccessible directly from internet
- Secure jump access validated

12.10 Advantages of Bastion Host

The implemented Bastion Host provided several advantages:

- Secure backend server access
- Controlled SSH communication
- Improved infrastructure security
- Better administrative management
- Production-grade access architecture

12.11 Summary

A Bastion Host was successfully implemented to provide secure SSH access to private EC2 instances inside the AWS infrastructure.

The implementation improved backend security, restricted direct internet exposure, and enabled secure administrative communication within the production-grade architecture.

SECTION 13 — SECURITY GROUP CONFIGURATION

13.1 Introduction

Security Groups were configured to control inbound and outbound traffic for AWS resources inside the infrastructure. Security Groups act as virtual firewalls that help secure EC2 instances, Application Load Balancers, and backend resources.

The implementation followed the principle of least privilege by allowing only required communication between application layers.

13.2 Objective

The objective of this implementation was to:

- Secure AWS resources using firewall rules
- Control inbound and outbound communication
- Restrict unauthorized access
- Enable secure communication between application layers

- Improve production infrastructure security

13.3 Security Group Overview

Security Groups are stateful virtual firewalls attached to AWS resources.

They control:

- Inbound traffic
- Outbound traffic

Security Groups automatically allow return traffic for established connections.

Separate Security Groups were implemented for different infrastructure layers to improve security and traffic isolation.

13.4 Implemented Security Groups

The following Security Groups were configured:

- Application Load Balancer Security Group
- Web Tier Security Group
- App Tier Security Group
- Bastion Host Security Group
- Database Security Group

13.5 ALB Security Group Configuration

The ALB Security Group allowed public web traffic.

Port	Protocol	Purpose
80	HTTP	Web Traffic
443	HTTPS	Secure Web Traffic

The ALB forwarded traffic to frontend web servers.

13.6 Web Tier Security Group Configuration

The Web Tier Security Group allowed traffic only from the Application Load Balancer.

Source	Port	Purpose
--------	------	---------

ALB Security Group	80	Frontend Communication
Bastion Host	22	SSH Access

This restricted direct public access to frontend EC2 instances.

13.7 App Tier Security Group Configuration

The App Tier Security Group allowed traffic only from the Web Tier.

Source	Port	Purpose
Web Tier	Application Port	Backend Communication
Bastion Host	22	SSH Access

Backend servers remained private and secure.

13.8 Database Security Group Configuration

The Database Security Group allowed database communication only from the App Tier.

Source	Port	Purpose
App Tier	3306	MySQL Communication

This prevented direct database access from external users.

13.9 Traffic Flow

The communication flow was implemented as follows:

- Users access Application Load Balancer
- ALB forwards traffic to Web Tier
- Web Tier communicates with App Tier
- App Tier communicates with Database Tier

Each layer communicated only with authorized resources.

13.10 Security Best Practices Implemented

The following best practices were followed:

Least Privilege Access

Only required ports and communication paths were allowed.

Layered Security

Separate Security Groups were implemented for each infrastructure layer.

Controlled SSH Access

SSH access was restricted through the Bastion Host.

Backend Isolation

Backend and database resources were protected inside private subnets.

13.11 Validation

The following validations were performed successfully:

- HTTP and HTTPS access verified
- ALB to Web Tier communication verified
- Web Tier to App Tier communication verified
- Database communication verified
- Unauthorized access restricted successfully

13.12 Advantages of Security Groups

The implemented Security Groups provided several advantages:

- Improved infrastructure security
- Controlled communication
- Reduced attack surface
- Secure application deployment
- Production-grade firewall implementation

13.13 Summary

Security Groups were successfully configured to control secure communication between infrastructure components inside the AWS environment.

The implementation improved infrastructure protection, restricted unauthorized access, and established secure layered communication within the production-grade architecture.

SECTION 14 — STATIC WEBSITE DEPLOYMENT USING NGINX

14.1 Introduction

NGINX was installed and configured on EC2 instances to host static web applications inside the AWS infrastructure. The deployment was implemented to provide frontend web services and validate traffic flow through the Application Load Balancer.

Static websites were deployed across multiple EC2 instances to support high availability and load balancing.

14.2 Objective

The objective of this implementation was to:

- Deploy static web applications on EC2 instances
- Configure NGINX web server
- Validate frontend traffic flow
- Support load balancing architecture
- Implement production-style web hosting

14.3 NGINX Overview

NGINX is a high-performance web server used for:

- Hosting web applications
- Reverse proxy implementation
- Load balancing
- Traffic management

NGINX is widely used in production environments because of its:

- Performance
- Scalability
- Reliability
- Lightweight architecture

14.4 Implemented Components

The following components were implemented:

- NGINX Installation

- Static HTML Deployment
- Multi-EC2 Deployment
- Load Balancer Integration

14.5 NGINX Installation

NGINX was installed on frontend EC2 instances using the following commands:

```
sudo apt update  
sudo apt install nginx -y
```

After installation, the NGINX service was started and enabled.

14.6 Website Deployment

Static HTML pages were deployed inside the NGINX web directory.

Example directory:

```
/var/www/html/
```

Separate application pages were created for:

- App1
- App2

This helped validate path-based and host-based routing.

14.7 NGINX Validation

The NGINX service was verified using:

```
sudo systemctl status nginx
```

Browser access and curl commands were also used to validate application accessibility.

Example:

```
curl localhost
```

14.8 Multi-EC2 Deployment

Static applications were deployed across multiple EC2 instances inside different Availability Zones.

Benefits included:

- High Availability
- Load Distribution
- Fault Tolerance

The Application Load Balancer distributed traffic across these instances.

14.9 Traffic Flow

The traffic flow was implemented as follows:

- Users access applications through the Application Load Balancer
- ALB forwards requests to NGINX web servers
- NGINX serves static application content
- Responses are returned to users

This architecture improved application availability and scalability.

14.10 Security Configuration

The following security measures were implemented:

- HTTP/HTTPS access through ALB
- Restricted direct instance access
- Security Group-based traffic control
- Backend isolation

Only required ports were allowed for communication.

14.11 Validation

The following validations were performed successfully:

- NGINX installation verified

- Web pages accessible successfully
- ALB traffic forwarding verified
- Multi-instance deployment validated
- Static application hosting confirmed

14.12 Benefits of NGINX Deployment

The implemented NGINX deployment provided several advantages:

- Lightweight web hosting
- High performance
- Scalable frontend hosting
- Easy integration with ALB
- Production-grade application deployment

14.13 Summary

NGINX was successfully deployed on EC2 instances to host static web applications inside the AWS infrastructure.

The implementation validated frontend traffic flow, load balancing functionality, and scalable web hosting within the production-grade architecture.

SECTION 15 — THREE-TIER ARCHITECTURE IMPLEMENTATION

15.1 Introduction

A production-grade three-tier architecture was implemented inside AWS to separate the infrastructure into frontend, backend, and database layers. This architecture improves security, scalability, availability, and application management.

The three-tier architecture is widely used in enterprise environments because it isolates different application components and enables secure communication between layers.

15.2 Objective

The objective of this implementation was to:

- Separate application layers securely
- Improve infrastructure scalability

- Implement high availability architecture
- Restrict direct backend access
- Build production-grade cloud infrastructure

15.3 Three-Tier Architecture Overview

The implemented architecture consisted of three layers:

Web Tier

Handles frontend user requests and serves web content.

Application Tier

Processes backend business logic and internal application communication.

Database Tier

Stores application data securely.

Each layer was isolated using separate subnets and security configurations.

15.4 Architecture Flow

The application flow was implemented as follows:

User → Application Load Balancer → Web Tier → App Tier → Database Tier

This layered communication improved infrastructure security and scalability.

15.5 Web Tier Implementation

The Web Tier was deployed inside public subnets.

Components used:

- Application Load Balancer
- Frontend EC2 Instances
- NGINX Web Server

Responsibilities:

- Handle incoming user traffic
- Serve frontend applications
- Forward requests to backend servers

15.6 Application Tier Implementation

The Application Tier was deployed inside private subnets.

Responsibilities:

- Process backend logic
- Handle internal application requests
- Communicate with database layer

The App Tier was inaccessible directly from the internet.

15.7 Database Tier Implementation

The Database Tier was deployed inside private subnets.

Responsibilities:

- Store application data
- Handle database communication
- Support secure backend operations

The Database Tier accepted communication only from the Application Tier.

15.8 Security Implementation

Security was implemented using:

Public and Private Subnets

Frontend resources were deployed inside public subnets, while backend and database resources were deployed inside private subnets.

Security Groups

Traffic was controlled between:

- ALB and Web Tier
- Web Tier and App Tier
- App Tier and Database Tier

Bastion Host

Administrative SSH access to private instances was provided through the Bastion Host.

15.9 High Availability Implementation

The architecture was deployed across multiple Availability Zones.

Benefits included:

- Fault Tolerance
- Continuous Availability
- Better Reliability

If one Availability Zone failed, traffic was redirected to resources running in another Availability Zone.

15.10 Scalability Implementation

Scalability was supported using:

- Application Load Balancer
- Multiple EC2 Instances
- Auto Scaling readiness

This architecture allowed infrastructure expansion based on application traffic.

15.11 Traffic Flow

The communication flow was implemented as follows:

1. User sends request through internet
2. Route53 resolves the domain
3. Application Load Balancer receives traffic
4. ALB forwards traffic to frontend servers
5. Frontend servers communicate with backend servers

6. Backend servers communicate with database servers
7. Response is returned to the user

This architecture maintained secure layered communication.

15.12 Validation

The following validations were performed successfully:

- Frontend application accessibility verified
- Backend communication verified
- Database communication verified
- Multi-tier communication validated
- Load balancing verified
- High availability deployment confirmed

15.13 Benefits of Three-Tier Architecture

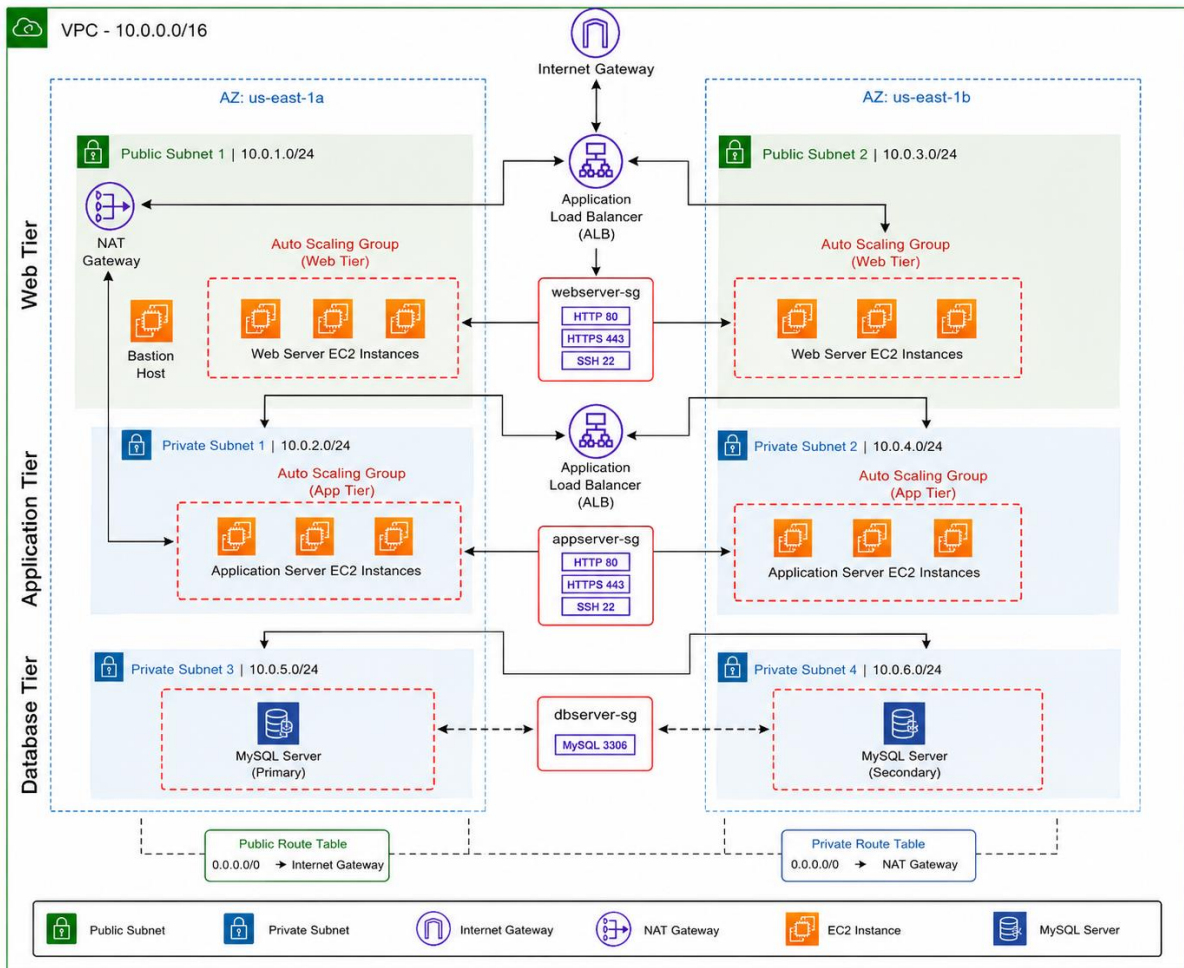
The implemented architecture provided several advantages:

- Better security
- Layer isolation
- Scalability
- High Availability
- Easier infrastructure management
- Production-grade deployment

15.14 Summary

A production-grade three-tier architecture was successfully implemented using frontend, backend, and database layers inside AWS infrastructure.

The implementation improved security, scalability, high availability, and application management while establishing secure communication between infrastructure components.



SECTION 16 — APPLICATION LOAD BALANCER IMPLEMENTATION

16.1 Introduction

An Application Load Balancer (ALB) was implemented to distribute incoming application traffic across multiple EC2 instances. The ALB improved scalability, high availability, and fault tolerance within the AWS infrastructure.

The Application Load Balancer was deployed inside public subnets and configured to forward traffic to frontend web servers running inside multiple Availability Zones.

16.2 Objective

The objective of this implementation was to:

- Distribute incoming traffic across multiple EC2 instances
- Improve application availability
- Implement fault-tolerant architecture
- Support scalable application deployment

- Enable advanced routing mechanisms

16.3 Application Load Balancer Overview

Application Load Balancer is a Layer 7 AWS load balancing service that distributes HTTP and HTTPS traffic.

ALB supports:

- Path-Based Routing
- Host-Based Routing
- HTTPS Termination
- Health Checks
- Traffic Distribution

The ALB forwards traffic only to healthy backend instances.

16.4 Implemented Components

The following components were configured:

- Application Load Balancer
- Target Groups
- Listener Rules
- Health Checks
- Multi-AZ Deployment

16.5 ALB Deployment Architecture

The Application Load Balancer was deployed inside public subnets across multiple Availability Zones.

Frontend EC2 instances were registered inside Target Groups.

Traffic flow:

User → Application Load Balancer → Target Group → EC2 Instances

This architecture improved application availability and scalability.

16.6 Target Group Configuration

Target Groups were created to route traffic to frontend EC2 instances.

The ALB forwarded requests to healthy instances registered inside the Target Groups.

Health checks continuously monitored instance health status.

16.7 Listener Configuration

The following listeners were configured:

Port	Protocol	Purpose
80	HTTP	Web Traffic
443	HTTPS	Secure Web Traffic

Listener rules controlled how incoming traffic was routed to backend resources.

16.8 Health Checks

Health checks were configured to monitor EC2 instance availability.

The ALB automatically stopped sending traffic to unhealthy instances.

Benefits included:

- Improved availability
- Automatic failure detection
- Better reliability

16.9 Multi-AZ Deployment

The ALB distributed traffic across EC2 instances deployed in multiple Availability Zones.

Benefits included:

- High Availability
- Fault Tolerance
- Load Distribution

If one Availability Zone failed, traffic continued through healthy instances in another Availability Zone.

16.10 Security Configuration

Security was implemented using Security Groups.

The ALB Security Group allowed:

- HTTP traffic (Port 80)
- HTTPS traffic (Port 443)

Backend EC2 instances accepted traffic only from the ALB Security Group.

This prevented direct public access to application servers.

16.11 Traffic Flow

The communication flow was implemented as follows:

1. User sends request through internet
2. Request reaches Application Load Balancer
3. ALB checks listener rules
4. ALB forwards request to Target Group
5. Healthy EC2 instance processes the request
6. Response is returned to the user

This architecture improved scalability and availability.

16.12 Validation

The following validations were performed successfully:

- ALB accessibility verified
- Traffic distribution verified
- Health checks validated
- Multi-instance communication verified
- High availability deployment confirmed

16.13 Benefits of Application Load Balancer

The implemented ALB provided several advantages:

- High Availability

- Fault Tolerance
- Scalable traffic handling
- Health monitoring
- Secure frontend traffic management
- Advanced routing support

16.14 Summary

An Application Load Balancer was successfully implemented to distribute incoming application traffic across multiple EC2 instances inside the AWS infrastructure.

The implementation improved application scalability, high availability, fault tolerance, and secure traffic management within the production-grade cloud architecture.

SECTION 17 — PATH-BASED ROUTING IMPLEMENTATION

17.1 Introduction

Path-Based Routing was implemented using the Application Load Balancer to route incoming traffic to different target groups based on URL paths. This implementation allowed multiple applications to run behind a single load balancer using different application paths.

The configuration improved resource utilization and simplified application management inside the AWS infrastructure.

17.2 Objective

The objective of this implementation was to:

- Route traffic based on URL paths
- Host multiple applications behind a single ALB
- Improve traffic management
- Simplify application deployment
- Support scalable architecture

17.3 Path-Based Routing Overview

Path-Based Routing is an Application Load Balancer feature that forwards traffic based on URL path patterns.

Example:

/app1 → Target Group 1

/app2 → Target Group 2

Different applications can be hosted using separate backend servers while sharing the same load balancer.

17.4 Implemented Components

The following components were configured:

- Application Load Balancer
- Multiple Target Groups
- Listener Rules
- Frontend EC2 Instances
- NGINX Application Paths

17.5 Application Paths Configured

The following paths were implemented:

Path	Target Group	Purpose
/app1	Target Group 1	Application 1
/app2	Target Group 2	Application 2

Each path forwarded traffic to different EC2 instances.

17.6 Listener Rule Configuration

Listener rules were configured inside the ALB.

Example routing flow:

User Request → ALB Listener → Path Rule → Target Group → EC2 Instance

The ALB checked the incoming URL path and forwarded traffic to the appropriate target group.

17.7 NGINX Configuration

NGINX was configured on EC2 instances to host multiple application paths.

Example directories:

```
/var/www/html/app1  
/var/www/html/app2
```

Separate static applications were deployed for path-based routing validation.

17.8 Traffic Flow

The communication flow was implemented as follows:

Application 1 Flow

domain.com/app1 → Target Group 1 → EC2 Instance 1

Application 2 Flow

domain.com/app2 → Target Group 2 → EC2 Instance 2

This architecture allowed multiple applications to share the same load balancer.

17.9 Security Implementation

Security was implemented using:

- ALB Security Group
- Backend Security Groups
- Controlled HTTP/HTTPS Access

Backend servers accepted traffic only from the Application Load Balancer.

17.10 Validation

The following validations were performed successfully:

- /app1 routing verified
- /app2 routing verified
- Target Group forwarding validated
- Listener rules verified
- Multi-application hosting confirmed

17.11 Benefits of Path-Based Routing

The implemented Path-Based Routing provided several advantages:

- Multiple application hosting
- Better resource utilization
- Simplified traffic management
- Scalable architecture
- Reduced infrastructure complexity

17.12 Summary

Path-Based Routing was successfully implemented using the Application Load Balancer to route traffic based on URL paths.

The implementation enabled multiple applications to run behind a single ALB while improving scalability, traffic management, and infrastructure efficiency inside the production-grade AWS architecture.

SECTION 18 — HOST-BASED ROUTING IMPLEMENTATION

18.1 Introduction

Host-Based Routing was implemented using the Application Load Balancer to route incoming traffic based on domain names. This implementation allowed multiple applications to be hosted behind a single load balancer using different hostnames.

The configuration improved application management, domain-based traffic handling, and enterprise-level routing architecture inside the AWS infrastructure.

18.2 Objective

The objective of this implementation was to:

- Route traffic based on domain names
- Host multiple applications using separate hostnames
- Improve traffic management
- Support scalable multi-domain architecture
- Implement enterprise-level routing

18.3 Host-Based Routing Overview

Host-Based Routing is an ALB feature that forwards traffic based on hostnames or domain names.

Example:

app1.domain.com → Target Group 1

app2.domain.com → Target Group 2

The ALB checks the hostname inside the incoming request and forwards traffic to the appropriate backend target group.

18.4 Implemented Components

The following components were configured:

- Application Load Balancer
- Multiple Target Groups
- Listener Rules
- Route53 DNS Records
- Frontend EC2 Instances

18.5 Domain Configuration

The following hostnames were implemented:

Hostname	Target Group	Purpose
app1.domain.com	Target Group 1	Application 1
app2.domain.com	Target Group 2	Application 2

Each domain forwarded traffic to different backend servers.

18.6 Listener Rule Configuration

Listener rules were configured inside the ALB based on host headers.

Example flow:

User Request → ALB Listener → Host Rule → Target Group → EC2 Instance

The ALB analyzed the hostname and routed traffic accordingly.

18.7 Route53 Integration

Route53 DNS records were configured to map domain names to the Application Load Balancer.

Example:

app1.domain.com → ALB DNS
app2.domain.com → ALB DNS

This enabled users to access applications using custom domain names.

18.8 Traffic Flow

The communication flow was implemented as follows:

Application 1 Flow

app1.domain.com → ALB → Target Group 1 → EC2 Instance 1

Application 2 Flow

app2.domain.com → ALB → Target Group 2 → EC2 Instance 2

This architecture supported multiple domain-based applications behind a single ALB.

18.9 Security Implementation

Security was implemented using:

- HTTPS communication
- ALB Security Group
- Backend Security Groups
- Controlled domain routing

Backend servers accepted traffic only from the Application Load Balancer.

18.10 Validation

The following validations were performed successfully:

- Domain routing verified
- Listener rules validated
- Route53 DNS mapping verified
- Backend communication confirmed
- Multi-domain hosting validated

18.11 Benefits of Host-Based Routing

The implemented Host-Based Routing provided several advantages:

- Multi-domain application hosting
- Simplified infrastructure
- Better traffic management
- Scalable architecture
- Enterprise-level domain routing

18.12 Summary

Host-Based Routing was successfully implemented using the Application Load Balancer and Route53 to route traffic based on domain names.

The implementation enabled scalable multi-domain application hosting while improving routing flexibility and infrastructure efficiency inside the production-grade AWS architecture.

SECTION 19 — SOURCE IP-BASED ROUTING IMPLEMENTATION

19.1 Introduction

Source IP-Based Routing was implemented using Application Load Balancer listener rules to control traffic access based on client IP addresses. This implementation allowed only authorized networks or users to access specific applications and services.

The configuration improved infrastructure security and supported enterprise-level access control within the AWS environment.

19.2 Objective

The objective of this implementation was to:

- Restrict application access based on source IP addresses
- Improve infrastructure security
- Implement enterprise-level traffic filtering
- Control user access to applications
- Support secure routing policies

19.3 Source IP-Based Routing Overview

Source IP-Based Routing allows traffic forwarding rules based on client IP ranges.

Example:

Allowed IP Range → Application Access

Blocked IP Range → Access Denied

The Application Load Balancer checks the source IP address before forwarding traffic to backend target groups.

19.4 Implemented Components

The following components were configured:

- Application Load Balancer
- Listener Rules
- Source IP Conditions
- Target Groups
- Security Groups

19.5 Listener Rule Configuration

Listener rules were configured with source IP conditions.

Example flow:

User Request → ALB Listener → Source IP Rule → Target Group

Only requests from authorized IP addresses were forwarded to backend applications.

19.6 Access Control Implementation

Access control policies were implemented for:

- Enterprise network access
- Restricted administrative access
- Controlled application exposure

Unauthorized source IP addresses were denied access automatically.

19.7 Traffic Flow

The communication flow was implemented as follows:

Authorized Access Flow

Authorized IP → ALB → Listener Rule → Target Group → EC2 Instance

Unauthorized Access Flow

Unauthorized IP → ALB → Access Denied

This architecture improved application security and controlled traffic access.

19.8 Security Implementation

Security improvements included:

IP-Based Filtering

Only trusted IP ranges were allowed.

Restricted Administrative Access

Sensitive applications were accessible only from authorized networks.

Reduced Attack Surface

Unauthorized external traffic was blocked before reaching backend servers.

19.9 Validation

The following validations were performed successfully:

- Authorized IP access verified
- Unauthorized access blocked
- Listener rule validation completed
- Secure traffic filtering confirmed

19.10 Benefits of Source IP-Based Routing

The implemented Source IP-Based Routing provided several advantages:

- Improved security
- Enterprise-level access control
- Restricted application exposure
- Better traffic filtering
- Reduced unauthorized access

19.11 Summary

Source IP-Based Routing was successfully implemented using Application Load Balancer listener rules to control traffic access based on client IP addresses.

The implementation improved infrastructure security, enabled enterprise-level access control, and supported secure traffic management within the production-grade AWS architecture.

SECTION 20 — ROUTE53 IMPLEMENTATION

20.1 Introduction

Amazon Route53 was implemented to provide DNS management and domain routing for applications deployed inside the AWS infrastructure. Route53 allowed users to access applications using domain names instead of IP addresses.

The implementation improved user accessibility, traffic routing, and integration with the Application Load Balancer.

20.2 Objective

The objective of this implementation was to:

- Implement DNS management
- Map domain names to AWS resources
- Improve user accessibility
- Integrate domains with Application Load Balancer
- Support scalable and highly available architecture

20.3 Route53 Overview

Amazon Route53 is a highly available and scalable DNS web service provided by AWS.

Route53 is used for:

- Domain registration
- DNS management
- Traffic routing
- Health checks

Route53 converts domain names into IP addresses or AWS resource endpoints.

20.4 Implemented Components

The following components were configured:

- Hosted Zones
- DNS Records
- Domain Mapping
- ALB Integration

20.5 Hosted Zone Configuration

A Hosted Zone was created for managing DNS records of the domain.

The Hosted Zone contained:

- A Records
- CNAME Records
- Alias Records

These records controlled domain routing inside AWS.

20.6 Domain Mapping

Domain names were mapped to the Application Load Balancer.

Example:

app1.domain.com → ALB DNS

app2.domain.com → ALB DNS

Users accessed applications using domain names instead of ALB URLs or IP addresses.

20.7 Route53 and ALB Integration

Route53 Alias Records were configured to point domain names directly to the Application Load Balancer.

Traffic flow:

User → Route53 → Application Load Balancer → EC2 Instances

This improved application accessibility and traffic management.

20.8 DNS Resolution Flow

The DNS communication flow was implemented as follows:

1. User enters domain name
2. Route53 receives DNS query
3. Route53 resolves the domain
4. Traffic is forwarded to ALB

5. ALB forwards requests to backend servers

This architecture simplified user access to applications.

20.9 High Availability Benefits

Route53 improved infrastructure availability through:

- Distributed DNS architecture
- Integration with multi-AZ ALB
- Scalable traffic routing
- Reliable DNS resolution

20.10 Security Implementation

Security improvements included:

Domain-Based Access

Applications were accessed using secure domain names.

Controlled Routing

Traffic routing was managed through Route53 and ALB integration.

HTTPS Support

Route53 supported secure HTTPS-based application access.

20.11 Validation

The following validations were performed successfully:

- Hosted Zone creation verified
- DNS record configuration verified
- Domain resolution validated
- Route53 to ALB routing confirmed
- Application accessibility through domain names verified

20.12 Benefits of Route53

The implemented Route53 configuration provided several advantages:

- Reliable DNS management
- Improved user accessibility
- Scalable domain routing
- High Availability
- Easy integration with AWS services

20.13 Summary

Amazon Route53 was successfully implemented to provide DNS management and domain routing inside the AWS infrastructure.

The implementation improved domain accessibility, traffic routing, high availability, and seamless integration with the Application Load Balancer within the production-grade architecture.

SECTION 21 — HTTPS & SSL/TLS IMPLEMENTATION

21.1 Introduction

HTTPS and SSL/TLS were implemented to provide secure communication between users and applications hosted inside the AWS infrastructure. HTTPS encrypts data transferred between clients and servers, improving application security and protecting sensitive information.

AWS Certificate Manager (ACM) and Application Load Balancer were used to configure secure HTTPS communication.

21.2 Objective

The objective of this implementation was to:

- Secure application communication
- Encrypt user traffic
- Implement HTTPS access
- Protect sensitive data transmission
- Improve infrastructure security

21.3 HTTPS Overview

HTTPS (HyperText Transfer Protocol Secure) is a secure version of HTTP.

HTTPS uses:

- SSL (Secure Sockets Layer)
- TLS (Transport Layer Security)

to encrypt communication between users and web applications.

HTTPS improves:

- Data security
- User trust
- Secure authentication
- Communication privacy

21.4 SSL/TLS Overview

SSL/TLS protocols are used to establish encrypted communication channels.

TLS is the modern and more secure version of SSL.

Functions of SSL/TLS include:

- Data encryption
- Secure communication
- Data integrity
- Authentication

This prevents attackers from reading sensitive communication data.

21.5 Implemented Components

The following components were configured:

- AWS Certificate Manager (ACM)
- SSL/TLS Certificate
- HTTPS Listener
- Application Load Balancer
- Secure Domain Communication

21.6 AWS Certificate Manager (ACM)

AWS Certificate Manager was used to generate and manage SSL/TLS certificates.

The ACM certificate was attached to the Application Load Balancer HTTPS listener.

Benefits of ACM:

- Free SSL/TLS certificates
- Automatic certificate renewal
- Easy AWS integration
- Simplified certificate management

21.7 HTTPS Listener Configuration

An HTTPS listener was configured on the Application Load Balancer.

Port	Protocol	Purpose
443	HTTPS	Secure Web Traffic

The ALB accepted secure HTTPS requests and forwarded traffic to backend target groups.

21.8 Secure Traffic Flow

The secure communication flow was implemented as follows:

User → HTTPS Request → ALB → Backend EC2 Instances

Traffic between users and the Application Load Balancer was encrypted using SSL/TLS.

21.9 SSL/TLS Encryption Process

The encryption process worked as follows:

1. User sends HTTPS request
2. ALB presents SSL/TLS certificate
3. Secure handshake is established
4. Encrypted communication begins
5. Traffic is forwarded securely

This ensured secure communication between users and the application.

21.10 Security Benefits

The HTTPS implementation improved infrastructure security through:

Encrypted Communication

Sensitive data was encrypted during transmission.

Secure Authentication

Users accessed applications through trusted secure connections.

Data Integrity

Communication data was protected from tampering.

Improved User Trust

Applications displayed secure HTTPS access.

21.11 Validation

The following validations were performed successfully:

- HTTPS access verified
- ACM certificate attachment verified
- SSL/TLS communication validated
- Secure browser access confirmed
- ALB HTTPS listener verified

21.12 Benefits of HTTPS and SSL/TLS

The implemented HTTPS architecture provided several advantages:

- Secure communication
- Encrypted traffic
- Improved application security
- Better user trust
- Production-grade secure architecture

21.13 Summary

HTTPS and SSL/TLS were successfully implemented using AWS Certificate Manager and Application Load Balancer to secure application communication inside the AWS infrastructure.

The implementation improved traffic security, protected sensitive data transmission, and established secure production-grade communication within the cloud architecture.

SECTION 22 — SSL TERMINATION & TLS COMMUNICATION

22.1 Introduction

SSL Termination was implemented at the Application Load Balancer to handle secure HTTPS communication efficiently within the AWS infrastructure. The ALB managed SSL/TLS decryption before forwarding requests to backend servers.

This implementation simplified certificate management, improved performance, and established secure frontend communication for the production-grade architecture.

22.2 Objective

The objective of this implementation was to:

- Handle SSL/TLS communication securely
- Implement SSL termination at ALB
- Reduce backend processing overhead
- Simplify certificate management
- Improve secure traffic handling

22.3 SSL Termination Overview

SSL Termination is the process where encrypted HTTPS traffic is decrypted at the Load Balancer instead of backend servers.

In this architecture:

- Users communicate securely with the ALB using HTTPS
- ALB decrypts SSL/TLS traffic
- Requests are forwarded to backend EC2 instances

This reduces the SSL processing workload on backend servers.

22.4 Implemented Components

The following components were configured:

- Application Load Balancer
- HTTPS Listener
- ACM SSL/TLS Certificate
- Backend Target Groups
- Secure Traffic Routing

22.5 TLS Communication Flow

The secure communication flow was implemented as follows:

User → HTTPS Request → ALB (SSL Termination) → EC2 Instances

The ALB handled encryption and decryption processes before forwarding requests internally.

22.6 SSL/TLS Handshake Process

The TLS communication process worked as follows:

1. User sends HTTPS request
2. ALB presents SSL/TLS certificate
3. TLS handshake is established
4. Secure encrypted session is created
5. ALB decrypts incoming traffic
6. Request is forwarded to backend servers

This ensured secure frontend communication.

22.7 Benefits of SSL Termination

SSL Termination provided several advantages:

Reduced Backend Load

Backend servers did not process SSL encryption and decryption.

Centralized Certificate Management

SSL/TLS certificates were managed only at the ALB.

Improved Performance

ALB handled secure communication efficiently.

Simplified Architecture

Backend servers focused only on application processing.

22.8 Security Implementation

Security improvements included:

Encrypted Frontend Communication

Users communicated securely using HTTPS.

Secure Certificate Management

ACM managed SSL/TLS certificates securely.

Controlled Traffic Flow

ALB securely handled incoming traffic before forwarding requests internally.

22.9 Validation

The following validations were performed successfully:

- HTTPS communication verified
- SSL certificate validation confirmed
- TLS handshake verified
- Secure browser access validated
- ALB SSL termination confirmed

22.10 High Availability Benefits

SSL Termination at ALB improved:

- Scalable secure traffic handling
- Fault-tolerant communication
- Multi-AZ secure architecture

- Reliable frontend connectivity

22.11 Summary

SSL Termination and TLS communication were successfully implemented using the Application Load Balancer and AWS Certificate Manager.

The implementation improved secure communication, simplified certificate management, reduced backend processing overhead, and established production-grade HTTPS traffic handling within the AWS infrastructure.

SECTION 23 — ROUTE53 + ALB + HTTPS INTEGRATION

23.1 Introduction

Route53, Application Load Balancer, and HTTPS services were integrated to provide secure, scalable, and highly available application access inside the AWS infrastructure.

This integration enabled users to access applications securely using domain names over HTTPS while the Application Load Balancer distributed traffic across multiple backend EC2 instances.

23.2 Objective

The objective of this implementation was to:

- Integrate Route53 with Application Load Balancer
- Enable secure HTTPS communication
- Implement domain-based application access
- Improve application availability
- Build production-grade secure traffic flow

23.3 Integrated Components

The following AWS services were integrated:

- Amazon Route53
- Application Load Balancer
- AWS Certificate Manager (ACM)
- HTTPS Listener

- EC2 Instances
- Target Groups

23.4 Architecture Flow

The integrated traffic flow was implemented as follows:

User → Route53 → HTTPS ALB → Target Group → EC2 Instances

This architecture provided secure and scalable application communication.

23.5 Route53 DNS Resolution

The communication process started with Route53 DNS resolution.

Steps:

1. User enters domain name
2. Route53 receives DNS request
3. Route53 resolves domain to ALB DNS
4. Traffic is forwarded to Application Load Balancer

This enabled users to access applications using custom domain names.

23.6 HTTPS Communication

The Application Load Balancer was configured with an HTTPS listener using ACM SSL/TLS certificates.

Port	Protocol	Purpose
443	HTTPS	Secure Web Traffic

All frontend communication between users and ALB was encrypted securely.

23.7 Traffic Distribution

After HTTPS communication was established:

- ALB checked listener rules
- Requests were forwarded to Target Groups

- Healthy EC2 instances processed the requests
- Responses were returned securely to users

The ALB distributed traffic across multiple Availability Zones for high availability.

23.8 High Availability Implementation

The architecture supported high availability through:

- Multi-AZ deployment
- Load balancing
- Health checks
- Distributed DNS routing

If one EC2 instance or Availability Zone failed, traffic was automatically routed to healthy resources.

23.9 Security Implementation

Security improvements included:

HTTPS Encryption

Frontend communication was encrypted using SSL/TLS.

Secure DNS Routing

Route53 managed secure domain resolution.

Backend Isolation

Backend EC2 instances remained protected behind the ALB.

Controlled Traffic Access

Security Groups restricted unauthorized communication.

23.10 Validation

The following validations were performed successfully:

- Route53 DNS resolution verified
- HTTPS communication validated

- ACM certificate verification completed
- ALB integration confirmed
- Secure application accessibility verified

23.11 Benefits of Integration

The integrated architecture provided several advantages:

- Secure HTTPS communication
- Scalable traffic management
- High Availability
- Fault Tolerance
- Reliable DNS routing
- Production-grade architecture

23.12 Summary

Route53, Application Load Balancer, and HTTPS services were successfully integrated to provide secure and highly available application access inside the AWS infrastructure.

The implementation improved secure communication, scalable traffic handling, DNS accessibility, and production-grade cloud architecture deployment.

SECTION 24 — AUTO SCALING GROUP IMPLEMENTATION

24.1 Introduction

Auto Scaling Groups (ASG) were implemented to automatically manage EC2 instance scaling based on application traffic and workload. The implementation improved scalability, high availability, and fault tolerance within the AWS infrastructure.

Auto Scaling automatically launched or terminated EC2 instances depending on traffic demand, ensuring optimal resource utilization and application performance.

24.2 Objective

The objective of this implementation was to:

- Implement automatic infrastructure scaling
- Improve application availability

- Handle traffic fluctuations dynamically
- Maintain application performance
- Support production-grade scalable architecture

24.3 Auto Scaling Overview

AWS Auto Scaling automatically adjusts the number of EC2 instances based on application workload.

Auto Scaling helps:

- Increase instances during high traffic
- Reduce instances during low traffic
- Maintain application availability
- Improve infrastructure efficiency

The Auto Scaling Group continuously monitors instance health and traffic conditions.

24.4 Implemented Components

The following components were configured:

- Launch Template
- Auto Scaling Group
- Scaling Policies
- Health Checks
- Application Load Balancer Integration

24.5 Launch Template Configuration

A Launch Template was created to define EC2 instance configuration.

The Launch Template included:

- Amazon Machine Image (AMI)
- Instance Type
- Security Groups
- Key Pair
- User Data Configuration

This template was used for automatic EC2 instance creation.

24.6 Auto Scaling Group Configuration

The Auto Scaling Group was configured with:

Configuration	Value
Minimum Instances	2
Desired Instances	2
Maximum Instances	4

The ASG maintained the required number of healthy EC2 instances automatically.

24.7 Scaling Policy Configuration

Scaling policies were configured to increase or decrease EC2 instances based on workload.

Example conditions:

- High CPU Usage → Scale Out
- Low CPU Usage → Scale In

This enabled dynamic resource scaling.

24.8 Health Checks

Health checks continuously monitored EC2 instance availability.

If an instance became unhealthy:

- ASG terminated the unhealthy instance
- A new healthy instance was launched automatically

This improved application reliability and fault tolerance.

24.9 ALB Integration

The Auto Scaling Group was integrated with the Application Load Balancer.

Traffic flow:

User → ALB → Auto Scaling EC2 Instances

The ALB distributed traffic across healthy EC2 instances created by the Auto Scaling Group.

24.10 High Availability Implementation

Auto Scaling improved high availability through:

- Multi-AZ deployment
- Automatic instance replacement
- Dynamic scaling
- Fault-tolerant infrastructure

If one instance failed, new instances were automatically created.

24.11 Scalability Benefits

The implemented Auto Scaling architecture provided:

Dynamic Scaling

Infrastructure scaled automatically based on demand.

Better Resource Utilization

Resources were optimized during varying traffic conditions.

Improved Performance

Applications maintained stable performance during high traffic.

Reduced Manual Management

Infrastructure scaling became automated.

24.12 Validation

The following validations were performed successfully:

- Launch Template creation verified
- Auto Scaling Group configuration verified
- Instance scaling validated
- Health checks confirmed
- ALB integration verified

24.13 Benefits of Auto Scaling

The implemented Auto Scaling architecture provided several advantages:

- Scalability
- High Availability
- Fault Tolerance
- Automatic infrastructure management
- Production-grade deployment

24.14 Summary

Auto Scaling Groups were successfully implemented to dynamically manage EC2 infrastructure based on application workload and traffic conditions.

The implementation improved scalability, application availability, fault tolerance, and automated infrastructure management within the production-grade AWS architecture.

SECTION 25 — PRODUCTION BEST PRACTICES

25.1 Introduction

Production best practices were followed throughout the AWS infrastructure implementation to improve security, scalability, high availability, and overall infrastructure reliability.

These practices helped build a production-grade cloud architecture suitable for enterprise environments.

25.2 Multi-AZ Deployment

Resources were deployed across multiple Availability Zones to improve high availability and fault tolerance.

Benefits included:

- Reduced downtime
- Continuous application availability
- Better disaster recovery
- Improved reliability

If one Availability Zone failed, resources in another zone continued serving traffic.

25.3 Public and Private Subnet Segmentation

Infrastructure resources were separated using public and private subnets.

Public Subnets

Used for:

- Application Load Balancer
- Bastion Host
- Frontend Web Servers

Private Subnets

Used for:

- Backend Application Servers
- Database Layer

This improved infrastructure security and network isolation.

25.4 Secure Internet Communication

Secure internet communication was implemented using:

- HTTPS
- SSL/TLS
- AWS Certificate Manager
- Application Load Balancer

Frontend traffic between users and the infrastructure was encrypted securely.

25.5 Application Load Balancer Usage

Application Load Balancer was implemented to improve:

- Traffic distribution
- High availability
- Fault tolerance
- Scalable traffic handling

ALB distributed requests across multiple healthy EC2 instances.

25.6 Auto Scaling Implementation

Auto Scaling Groups were implemented to dynamically manage infrastructure scaling.

Benefits included:

- Automatic scaling
- Better resource utilization
- Improved application performance
- Automated recovery

This supported scalable production architecture.

25.7 Security Group Restrictions

Security Groups were configured using least exposure principles.

Only required ports and communication paths were allowed.

Examples:

- HTTP → Port 80
- HTTPS → Port 443
- SSH → Port 22

Backend communication was restricted to authorized resources only.

25.8 NAT Gateway Usage

NAT Gateway was implemented to provide outbound internet communication for private resources without exposing them directly to the internet.

This improved:

- Backend security
- Controlled internet access
- Private subnet isolation

25.9 Route53 DNS Integration

Route53 was integrated for domain-based application access.

Benefits included:

- DNS management
- Better user accessibility
- Reliable domain resolution
- ALB integration

25.10 Health Checks and Monitoring

Health checks were implemented using:

- Application Load Balancer
- Auto Scaling Groups

Unhealthy instances were automatically detected and replaced.

This improved:

- Application reliability
- Fault tolerance
- Infrastructure stability

25.11 High Availability Architecture

High availability was implemented using:

- Multi-AZ deployment
- Load Balancer
- Auto Scaling
- Health Checks

This ensured continuous application accessibility during failures.

25.12 Scalability Architecture

Scalability was implemented using:

- Multiple EC2 Instances
- Application Load Balancer
- Auto Scaling Groups

The infrastructure automatically handled increasing traffic loads.

25.13 Secure Traffic Flow

The secure communication flow was implemented as follows:

User → Route53 → HTTPS ALB → Web Tier → App Tier → Database Tier

This architecture maintained secure and scalable communication throughout the infrastructure.

25.14 Benefits of Production Best Practices

The implemented best practices provided several advantages:

- Improved security
- High availability
- Scalability
- Fault tolerance
- Better infrastructure management
- Production-grade deployment

25.15 Summary

Production best practices were successfully implemented throughout the AWS infrastructure to improve security, scalability, reliability, and high availability.

The implementation established a secure and production-grade cloud architecture suitable for enterprise-level application deployment.

SECTION 26 — FINAL ARCHITECTURE SUMMARY

26.1 Introduction

A production-grade AWS cloud infrastructure was successfully designed and implemented using secure networking, scalable architecture, high availability, load balancing, DNS integration, and HTTPS communication.

The architecture was built using multiple AWS services integrated together to support secure and reliable application deployment.

26.2 Final Architecture Overview

The implemented infrastructure included:

- Amazon VPC
- Public and Private Subnets
- Internet Gateway
- NAT Gateway
- Route Tables
- EC2 Instances
- Bastion Host
- Security Groups
- NGINX Web Servers
- Application Load Balancer
- Auto Scaling Groups
- Route53
- HTTPS & SSL/TLS
- Multi-AZ Deployment

These services together formed a secure and scalable production-grade cloud architecture.

26.3 Final Architecture Flow

The final communication flow was implemented as follows:

User

- Route53
- HTTPS Application Load Balancer
- Web Tier EC2 Instances
- Application Tier EC2 Instances
- Database Tier

This architecture provided secure, scalable, and highly available application communication.

26.4 Networking Architecture Summary

Secure networking was implemented using:

- Custom Amazon VPC

- Public and Private Subnets
- Internet Gateway
- NAT Gateway
- Route Tables
- Security Groups

The infrastructure separated frontend and backend resources securely while maintaining controlled communication between application layers.

26.5 High Availability Summary

High availability was implemented using:

- Multi-AZ Deployment
- Application Load Balancer
- Health Checks
- Auto Scaling Groups

Traffic was automatically distributed across healthy EC2 instances deployed in multiple Availability Zones.

26.6 Scalability Summary

Scalability was implemented using:

- Multiple EC2 Instances
- Application Load Balancer
- Auto Scaling Groups
- Dynamic Scaling Policies

The infrastructure automatically handled increasing application traffic and workload demands.

26.7 Security Summary

Security was implemented using:

- HTTPS & SSL/TLS
- AWS Certificate Manager
- Security Groups
- Private Subnets

- Bastion Host
- Controlled Traffic Routing

This ensured secure frontend and backend communication inside the infrastructure.

26.8 DNS and Routing Summary

Route53 and Application Load Balancer were integrated to provide:

- DNS Management
- Domain-Based Routing
- Path-Based Routing
- Host-Based Routing
- Source IP-Based Routing

This improved traffic management and user accessibility.

26.9 Production Architecture Benefits

The implemented production-grade architecture provided several advantages:

- Secure networking
- High availability
- Scalability
- Fault tolerance
- Load balancing
- Secure communication
- Centralized traffic management
- Enterprise-level deployment architecture

26.10 Validation Summary

The following implementations were validated successfully:

- VPC communication
- Public and private subnet routing
- Internet Gateway communication
- NAT Gateway outbound access
- EC2 deployment
- NGINX hosting

- ALB traffic distribution
- Route53 DNS resolution
- HTTPS secure communication
- Auto Scaling functionality

26.11 Conclusion

The AWS infrastructure implementation successfully demonstrated a production-grade cloud architecture using core AWS networking, scalability, high availability, load balancing, DNS management, and secure HTTPS communication services.

The project provided practical understanding of real-world AWS infrastructure deployment and enterprise-level cloud architecture design using modern AWS services and best practices.

